Client-side Technologies

Eng. Niveen Nasr El-Den
SD & Gaming CoE
iTi

Day 6

Document Object Model

DOM

DOM

- The **document** object in the **BOM** is the top level of the **DOM** hierarchy.
- DOM is a representation of the whole document as nodes and attributes.
- You can access each of these nodes and attributes and change or remove them.
- You can also create new ones or add attributes to existing ones.
- DOM is a subset of BOM.
- In other word: **the document is yours!**

DOM

- DOM Stands for Document Object Model.
- W3C standard.
- Its an API that interact with documents like HTML, XML.. etc.
- Defines a standard way to access and manipulate HTML documents.
- Platform independent.
- Language independent

DOM

- The **document** object in the **BOM** is the top level of the **DOM** hierarchy.
- DOM is a representation of the whole document as nodes and attributes.
- You can access each of these nodes and attributes and change or remove them.
- You can also create new ones or add attributes to existing ones.

DOM is a subset of BOM.

In other word: **the document is yours!**

DOM Relationships

Scripting HTML

HTML DOM

- The HTML DOM is a standard for how to get, change, add, or delete HTML elements.
- It is a hierarchy of data types for HTML documents, links, forms, comments, and everything else that can be represented in HTML code.
- The general data type for objects in the DOM are *Nodes*. They have *attributes*, and some nodes can contain other nodes.
- There are several node types, which represent more specific data types for HTML elements. Node types are represented by numeric constants.

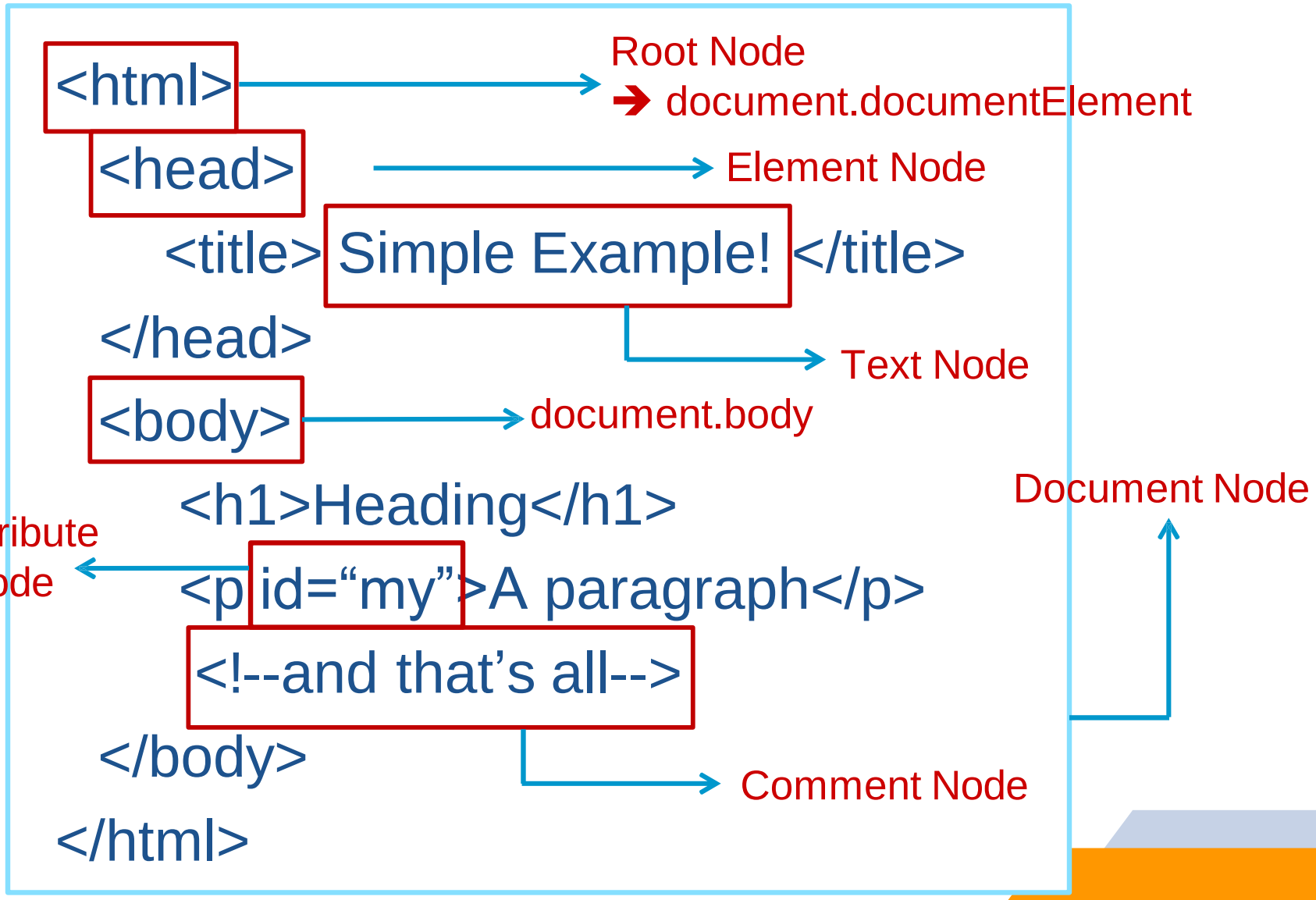
HTML DOM

- It allows code running in a browser to access and interact with every node in the document.
- Nodes can be created, moved and changed.
- Event listeners can be added to nodes and triggered on occurrence of a given event.

HTML DOM

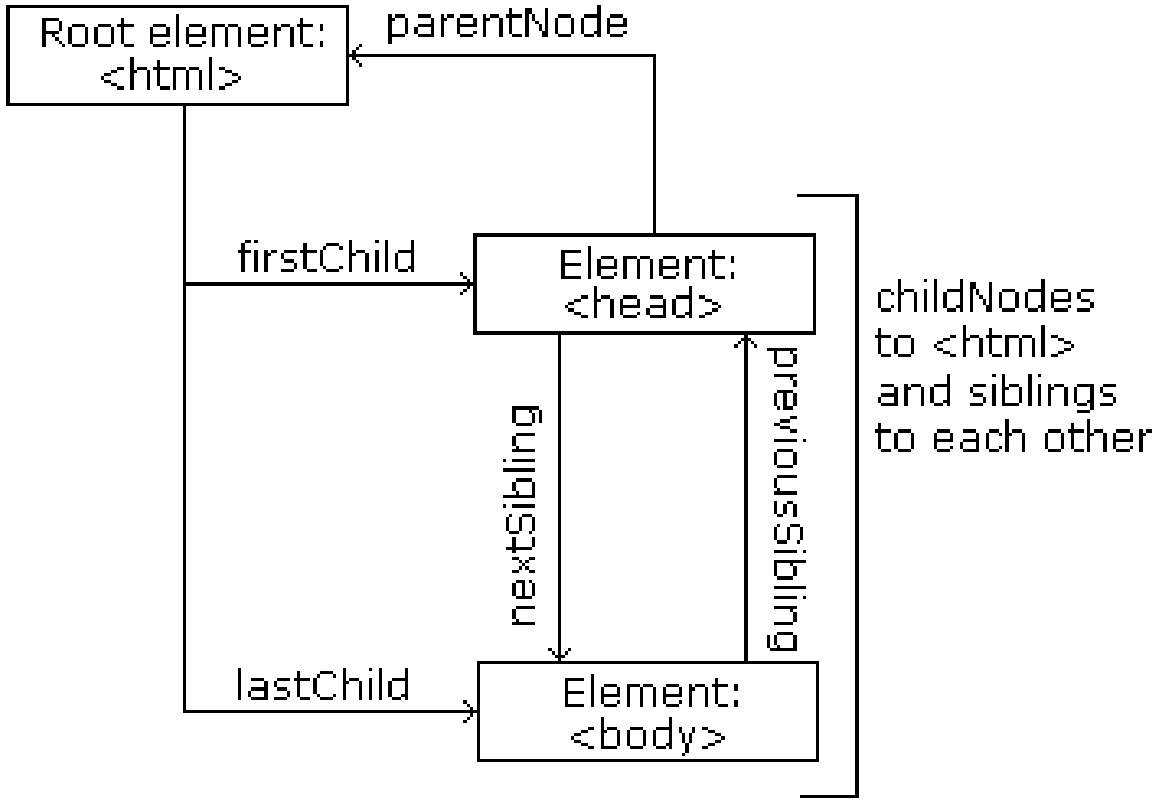
- According to the DOM, everything in an HTML document is a node.
- The DOM says:
  - The entire document is a document node
  - Every HTML element is an element node
  - The text in the HTML elements are text nodes
  - Every HTML attribute is an attribute node
  - Comments are comment nodes
- JavaScript is powerful DOM Manipulation

Simple Example!



Node Tree

- The HTML DOM views HTML document as a node-tree.
- All the nodes in the tree have relationships to each other.
  - Parent
    - parentNode
  - Children
    - firstChild
    - lastChild
  - Sibling
    - nextSibling
    - previousSibling

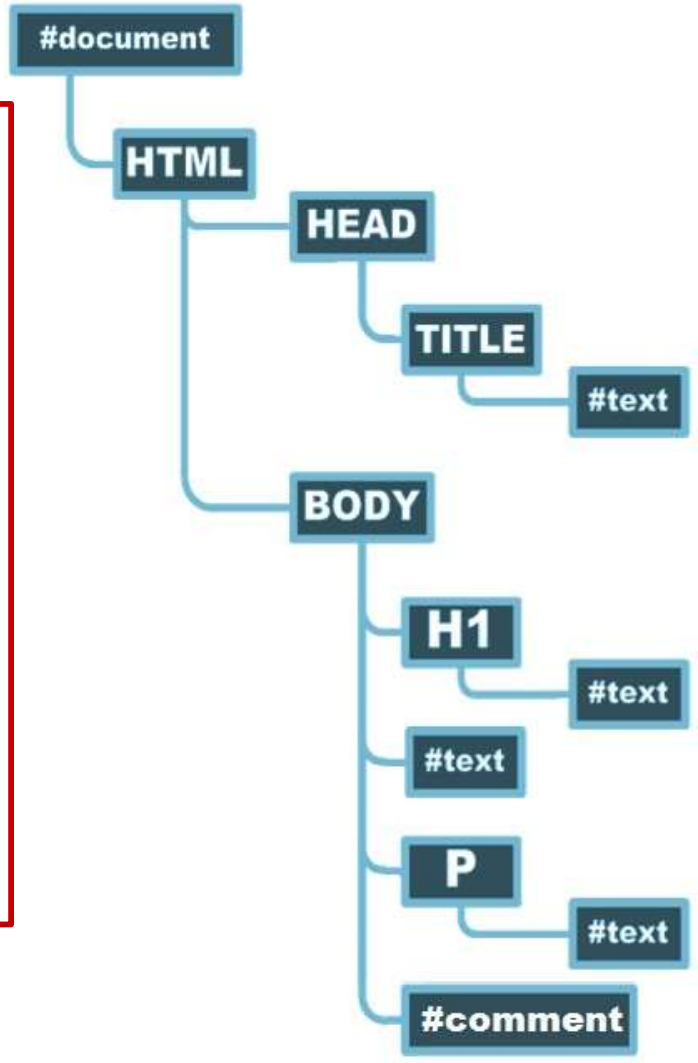


Nodes Relationships

- The terms **parent**, **child**, and **sibling** are used to describe the relationships.
  - Parent nodes have children.
  - Children on the same level are called siblings (brothers or sisters).
- Attribute nodes are not child nodes of the element they belong to, and have no parent or sibling nodes
- In a node tree, the top node is called the root
- Every node, except the root, has exactly one parent node
- A node can have any number of children
- A leaf is a node with no children
- Siblings are nodes with the same parent

Simple Example!

```
<html>
        <head>
            <title>Simple Example!</title>
        </head>
        <body>
            <h1>Greeting</h1>
             Welcome All
            <p>A paragraph</p>
            <!-- and that's all-->
        </body>
 </html>
```



Node Properties

- All nodes have three main properties

Property	Description
nodeName	Returns HTML Tag name in uppercase display
tagname	
nodeType	returns a numeric constant to determine node type. There are 12 node types.
nodeValue	returns null for all node types except for text and comment nodes.

Using nodeName
If node is text it returns #text
For comment it returns #comment
For document it returns #document

Value	Description
1	Element Node
2	Attribute Node
3	Text Node
8	Comment Node
9	Document Node

To get the Root Element: document.documentElement.

Node Collections

- Node Collections have One Property
  - **length : gives the length of the Collection.**
    - **e.g. childNodes.length:** returns number of elements inside the collection
- We can check if there is child collection using
  - **hasChildNodes():** Tells if a node has any children
- We can check if there is attribute collection using
  - **hasAttributes():** Tells if a node has any attributes

Collection	Description	Accessing
childNodes	Collection of element's children	childNodes[ ] childNodes.item()
attributes	Returns collection of the attributes of an element	attributes[ ] attributes.item()

Dealing With Nodes

- Dealing with nodes fall into four main categories:
  - Accessing Node
  - Modifying Node’s content
  - Adding New Node
  - Remove Node from tree

Accessing DOM Nodes

- You can access a node in 5 main ways:
  - [window.]document.getElementById(“id”)
  - [window.]document.getElementsByName(“name”)
  - [window.]document.getElementsByTagName(“tagname”)
  - By navigating the node tree, using the node relationships
  - New HTML5 Selectors.

Example!

New HTML5 Selectors

- In HTML5 we can select elements by ClassName

```
var elements = document.getElementsByClassName('entry');
```

- Moreover there's now possibility to fetch elements that match provided CSS syntax

```
var elements =  document.querySelectorAll("#someClasses)");
```

```
var first_td =   document.querySelector("span");
```

New HTML5 Selectors

- Selecting the first div met

```
var elements = document.querySelector("div");
```

- Selecting the first item with class SomeClass

```
var elements = document.querySelector(".SomeClass");
```

- Selecting the first item with id someID

```
var elements = document.querySelector("#someID");
```

- Selecting all the divs in the current container

```
var elements = document.querySelectorAll("div");
```

Accessing DOM Nodes

Navigating the node tree, using the node relationships

firstChild	**Move direct to first child**
lastChild	**Move direct to last child**
parentNode	**To access child's parent**
nextSibling	**Navigate down the tree one node step**
previousSibling	**Navigate up the tree one node step**
Using children collection → childNodes[ ]	

Example!

Modifying Node's Content

- Changing the Text Node by using

innerHTML	**Sets or returns the HTML contents (+text) of an element**
textContent	**Equivalent to innerText.**
nodeValue → with text and comment nodes only	
setAttribute()	**Modify/Adds a new attribute to an element**
just using attributes as object properties	

- Modifying Styles
  - Node.style

Example!

Creating & Adding Nodes

Method	Description
createElement()	To create new tag element
createTextNode()	To create new text element
createAttribute()	To creates an attribute element
createComment()	To creates an comment element

Creating & Adding Nodes

Method	Description
cloneNode(true|false)	Creating new node a copy of existing node. It takes a Boolean value true: Deep copy with all its children or false: Shallow copy only the node
b.appendChild(a)	To add new created node “a” to DOM Tree at the end of the selected element “b”.
insertBefore(a,b)	Similar to appendChild() with extra parameter, specifying before which element to insert the new node. a: the node to be inserted b: where a should be inserted before **document.body.insertBefore(a,b)**

Example!

Removing DOM Nodes

Method	Description
removeChild()	To remove node from DOM tree
parent.replaceChild(n,o)	To remove node from DOM tree and put another one in its place n: new child o: old child
removeAttribute()	Removes a specified attribute from an element

- A quick way to wipe out all the content of a subtree is to set the innerHTML to a blank string. This will remove all of the children of <body>

```
document.body.innerHTML="";
```

Example!

Summary

- Access nodes:
  - Using parent/child relationship properties parentNode, childNodes, firstChild, lastChild, nextSibling, previousSibling
  - Using getElementsById(), getElementsByTagName(), getElementsByName()
- Modify nodes:
  - Using innerHTML or innerText/textContent
  - Using nodeValue or setAttribute() or just using attributes as object properties
- Remove nodes with
  - removeChild() or replaceChild()
- And add new ones with
  - appendChild(), cloneNode(), insertBefore()

Dynamic HTML

the art of making dynamic and interactive web pages.

DHTML

- DHTML has no official definition or specification.
- DHTML stands for Dynamic HTML.
- DHTML is NOT a scripting language.
- DHTML is not w3c (i.e. not a standard).
- DHTML is a browser feature-that gives you the ability to make dynamic Web pages.
- "*Dynamic*" is defined as the ability of the browser to alter a web page's look and style *after* the document has been loaded.
- DHTML is very important in web development

DHTML

- DHTML uses a combination of:
  1. Scripting language
  2. DOM
  3. CSS

  to create HTML that can change even after a page has been loaded into a browser.

- DHTML is supported by 4.x generation browsers.

Assignment